Projet TPALT 2026

Breton Noé
n°21516014
Boudrouss Réda
n°28712638
Durbin Deniz Ali
n°21111116

Sorbonne Université
France

Introduction

RHETORICA est une plateforme de débat compétitif en temps réel, conçue selon le modèle de Chess.com appliqué au domaine de l'argumentation. Le projet répond à trois problèmes concrets : trouver un adversaire disponible sans attente excessive, obtenir un arbitrage impartial non soumis à la subjectivité humaine, et mesurer sa progression de façon objective.

La plateforme propose trois modes de jeu (entraînement contre intelligence artificielle, partie casuelle et partie classée), un système ELO (base 1200, K = 32), plus de 120 sujets en français et en anglais, ainsi que 24 succès débloquables. Chaque débat se déroule en 12 messages (6 par joueur, 90 secondes par tour) et chaque message est évalué en temps réel par le modèle Llama 3.3 70B via l'API Groq.

Une application déployé peut être trouvé sur **https ://tpalt.rboud.com** (mais nous avons désactivé l'IA).

Architecture Technique

Stack

Le projet repose sur **Next.js 15** (App Router, SSR, API Routes) pour le frontend et les endpoints REST, ainsi que sur un **serveur WebSocket personnalisé** (`server.ts`). La base de données est gérée par **Prisma**, avec SQLite en environnement de développement et PostgreSQL en production. L'inférence par intelligence artificielle s'appuie sur l'**API Groq** (Llama 3.3 70B). L'ensemble du projet est typé en **TypeScript 5** strict, avec TailwindCSS 4 et TanStack Query 5 côté client.

Le choix d'un serveur WebSocket développé en interne, plutôt que Socket.io, résulte de l'incompatibilité de Next.js avec les connexions longue durée mais aussi le besoin d'avoir une gestion du matchmaking qui soit rapide et réactive.

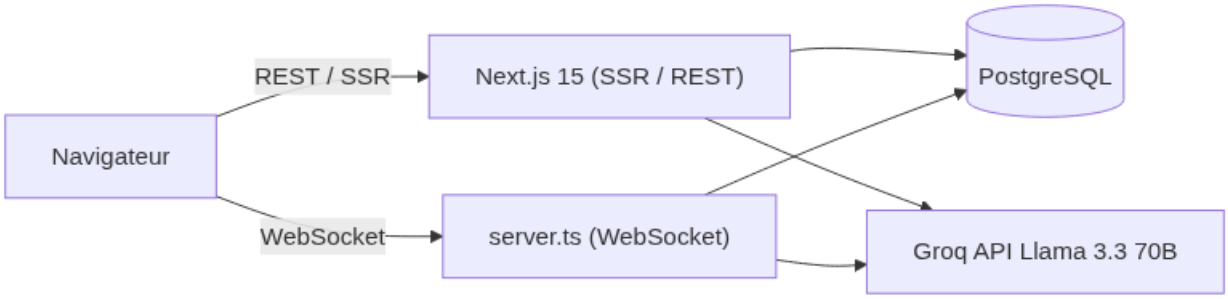

Figure 1 – Vue d'ensemble de l'architecture

Modèle de données

Le schéma de la base de donnée peut être retrouvé `prisma/schema.prisma`. Et un schéma de la db peut être retrouvé sur la figure 2.

WebSocket et cohérence

Le serveur websocket (`server.ts`) maintient en mémoire les salles actives et les files d'attente par mode de jeu. Afin de minimiser la latence perçue, les messages sont diffusés aux deux joueurs *avant* l'écriture en base de données. Un mécanisme de détection de connexions

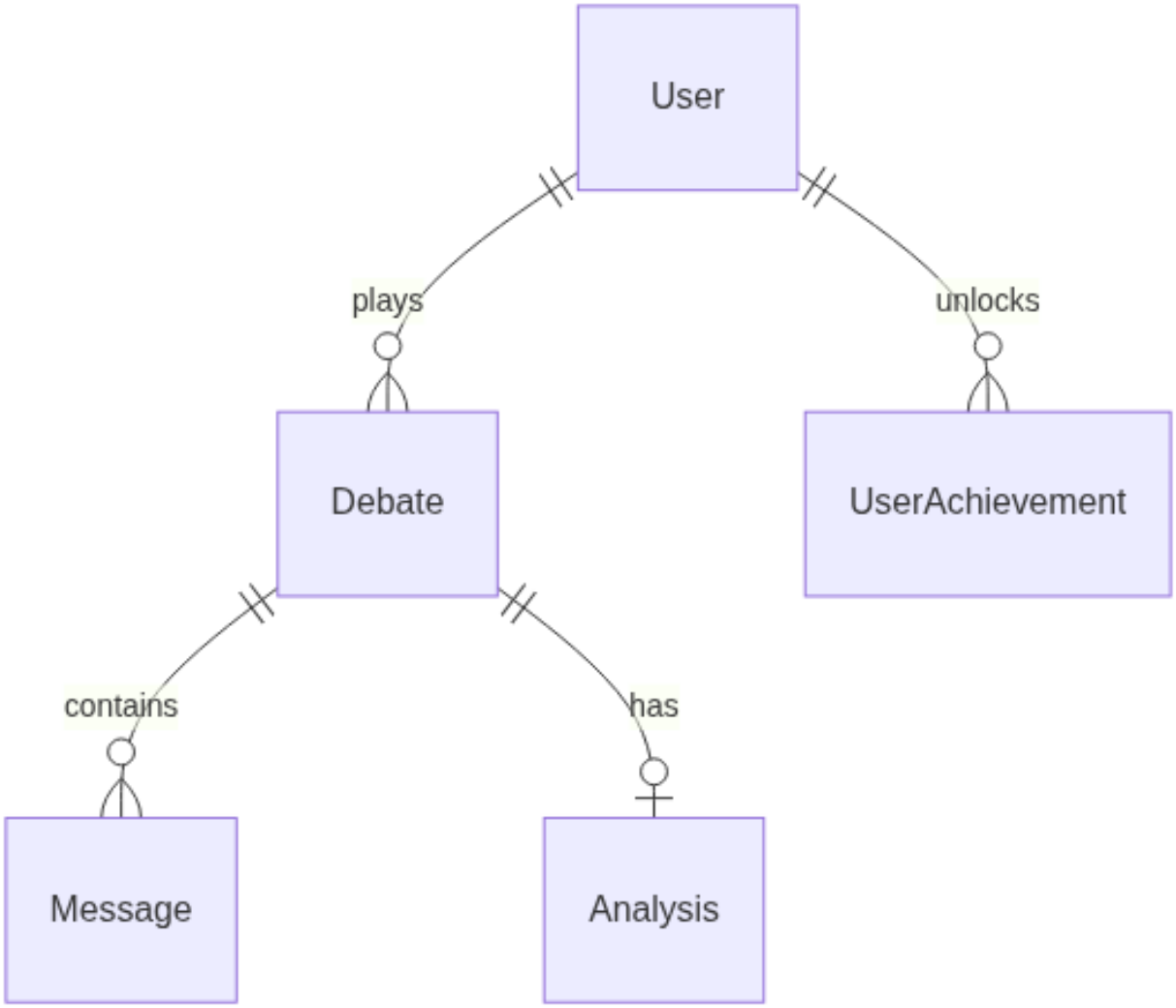

Figure 2 – schéma de base de donnée

inactives par échange ping/pong toutes les 15 secondes permet de libérer les connexions zombies.

Fonctionnalités Clés

Matchmaking et ELO

En mode casual, le matchmaking est réalisé selon une politique FIFO. En mode classé, la fenêtre ELO démarre à ±150 et s'élargit de ±50 toutes les 10 secondes (maximum ±500), afin d'éviter la famine de matchs aux extrémités du classement. Le calcul ELO suit la formule standard des échecs (K = 32, plancher = 100) : battre un adversaire mieux classé rapporte davantage de points.

La file d'attente est géré par le serveur websocket (`server.ts`) et la formule de calcule de l'élo peut être trouvé dans le fichier `src/lib/eli.ts`.

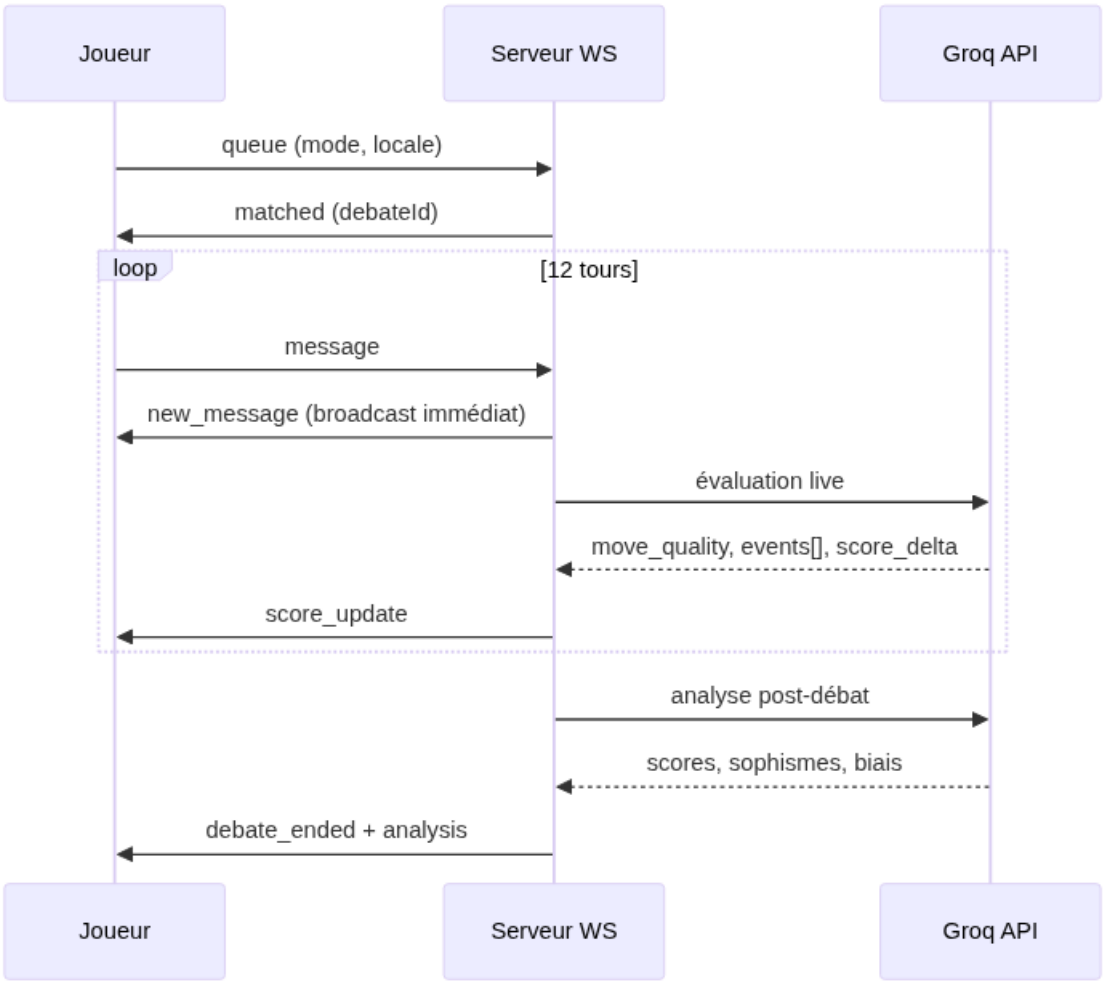

Figure 3 – Déroulement d'une partie

Intelligence Artificielle

Cinq prompts spécialisés couvrent l'ensemble du cycle de vie d'un débat :

Prompt	Déclenchement	Température
Évaluation live	Chaque message	0,1
Adversaire IA	Tour IA (entraînement)	0,9 / 0,7 / 0,4
Analyse post-débat	Fin de partie	0,3
Indice	Bouton « Indice »	0,5
Fact-check	À la demande	0,2

L'évaluation en temps réel détecte huit types d'événements (claim, evidence, counter_argument, sophism, ad_hominem, etc.) et calcule un delta de score par message. La règle critique *wrong_side* pénalise tout argument défendant la position adverse, quelle que soit sa qualité rhétorique.

L'analyse post-partie identifie les sophismes classiques (ad hominem, homme de paille, pente glissante, fausse dichotomie, etc.) avec citations exactes, et produit cinq scores individuels : qualité des arguments, style rhétorique, cohérence logique, fact-checking et score global.

L'adversaire IA propose trois niveaux de difficulté calibrés par température et par instructions système : facile (température 0,9, arguments informels et hors-sujet), moyen (température 0,7, thèse accompagnée de sa justification), difficile (température 0,4, contre-argument direct, structuré et sans répétition).

Les promptes peuvent être retrouvé dans le fichier `src/lib/prompts.ts`.

Anti-triche et succès

En mode classé, l'API Visibility du navigateur détecte les changements d'onglet, à partir de 3 infractions, une pénalité de 30 points ELO est appliquée. Le code est implémenté dans `src/app/(app)/debate/[id]/hooks/useAntiCheat.ts`

La plateforme propose 24 succès répartis en 6 catégories (Jalons, Victoires, Séries, Qualité, Progression, Ultime). On peut les retrouver dans le fichier `src/lib/achievements.ts`.

Déploiement et Perspectives

Pipeline CI/CD

L'application est conteneurisée via **Docker** (`Dockerfile`). Le pipeline CI/CD repose sur **Coolify** (PaaS auto-hébergé) : un push sur la branche `main` déclenche automatiquement une reconstruction et un redéploiement sur le serveur. Le trafic entrant transite par un **Cloudflare Tunnel** (masquage de l'adresse IP, protection DDoS et chiffrement SSL sans exposition de port), puis par **Traefik** (reverse proxy avec gestion automatique des certificats Let's Encrypt).

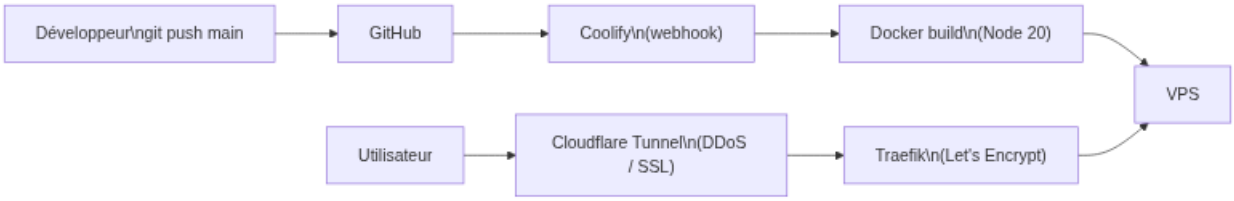

Figure 4 – Pipeline de déploiement continu

Perspectives d’évolution

Trois axes d’amélioration sont envisagés :

— l’ajout d’un **cache Redis** pour la gestion du classement et des sessions (afin de réduire la charge sur PostgreSQL)
— le **fine-tuning** du modèle sur des débats réels annotés (dans le but d’améliorer la précision de l’évaluation)
— l’implémentation de **replays commentés par l’IA** (permettant à l’utilisateur de revoir sa partie avec des annotations argumentatives générées a posteriori), qui est une grosse feature de chess.com.